Travel Constraint Extraction

Small-model fine-tuning for changes to a London itinerary

Hyojin Lee · Recent Areas of AI · Semester project · 29 October 2026

1. Task and research question

A traveler can express several changes in one sentence: keep a museum visit, return earlier, reduce walking, and choose an indoor alternative if it rains. Before an application can act, it must separate the requested conditions and preserve their meaning. This project treats that step as structured information extraction with a Transformer language model.

The input is an English change request together with a short reference itinerary. The output is a JSON list of constraints. The research question is whether supervised LoRA adaptation improves a 0.5-billion-parameter instruction model on this task, compared with the same model without adaptation and a larger model using two examples in its prompt. Routing stays deliberately simple so that the NLP component remains the main implementation and experiment.

An illustrative input and target

Reference: National Gallery, lunch, a South Bank walk, hotel return at 20:00.
Request: “Keep the gallery. I must be back by 19:00. I would prefer less walking.”

Extracted condition	Representation
Keep National Gallery	place / include / hard
Hotel return by 19:00	time / hotel_return / <= 19:00 / hard
Reduce walking	walking_amount / minimize / soft

This table explains the task; it is not a claimed model prediction. The executable schema also represents conditional weather actions, numeric walking limits, and unresolved values.

What the experiment establishes

The adapted model obtained content F1 0.795 and exact output on 57/80 held-out inputs. The baseline obtained F1 0.000, and the larger model obtained 0.782. These are results on a controlled, AI-reviewed synthetic dataset. They are not estimates of performance on real travelers or evidence of itinerary feasibility.

2. Dataset and annotation contract

The dataset contains 400 synthetic records: 100 reference scenarios, each paired with four request paraphrases. A record stores the reference itinerary, request, structured target, scenario identifier, split, and review provenance. Scenario groups remain intact in the 280/40/80 train/validation/test split. The test set contains 20 groups, two per task family.

Family	What the model must extract
Place inclusion / exclusion	Requested place and include/exclude operator
Hotel return time	Clock time and by / exactly / not-before relation
Walking limit / minimization	Numeric cap and scope, or qualitative preference
Ambiguous time / walking scope	Unresolved value or scope; clarification needed
Nearby lunch	Named or context-resolved anchor place
Weather alternative	Both rain and dry branches with their conditions
Combined request	Six conditions in one request

Review, duplication, and leakage controls

Version 1 contained identical request sentences across splits and an association between gallery identity and requirement strength. Before v2 model experiments, the data review removed 37 cross-split duplicate-request groups, introduced separate wording pools per split, and included both hard and soft requests for each gallery. V2 has 400 distinct full inputs and 252 distinct request texts. Intent families, semantic frames, and some wording within each split still repeat.

Review status: AI-assisted semantic review of authored patterns and instantiated values, plus automated checks on all 400 records. Zero records are marked human-reviewed. The reference labels therefore remain fallible. This is a transparent limitation of the submitted experiment, rather than an independent human annotation study.

Labeling rules

Direct instructions are hard requirements; “prefer” and “if possible” indicate soft preferences. “Less walking” is a soft minimization preference, not an invented numeric limit. Missing walking scope is null and requires clarification. Relative references such as “the gallery” and “the original return time” resolve against the supplied itinerary. Rain and dry alternatives are both extracted; later weather selection activates the relevant branch.

Frozen dataset SHA-256: 19466ef251eaa6fa81db7014e5ee9e180e95c71deb2b103e2b06f4df6e231aa5

3. Models and training

Qwen2.5 instruction models provide the Transformer backbone [1]. All approaches use the same schema and system instructions. The two small-model runs use the original bfloat16 0.5B checkpoint; the larger comparison uses an MLX 4-bit conversion of Qwen2.5-7B-Instruct. Model revisions are pinned in the experiment protocol.

Approach	Adaptation and prompt
0.5B baseline	Qwen2.5-0.5B-Instruct; zero-shot; no task training
0.5B + LoRA	Same base; task-specific supervised LoRA; zero-shot
7B two-shot	Qwen2.5-7B-Instruct, 4-bit; two training demonstrations; no task training

LoRA configuration

LoRA keeps pretrained weights fixed and learns low-rank updates [2]. This run updates 733,184 parameters in the last four Transformer layers. Prompt tokens are masked in the loss; supervised learning targets the annotated JSON continuation.

Setting	Value	Setting	Value
Training / validation	280 / 40	Optimizer	Adam
Steps / epochs	840 / 3	Learning rate	0.0001
Batch size / seed	1 / 42	LoRA rank / scale	8 / 20
Sequence cap	1,024 tokens	LoRA dropout	0.0
Max observed length	966 tokens	Trainable parameters	733,184

Execution and checkpoint verification

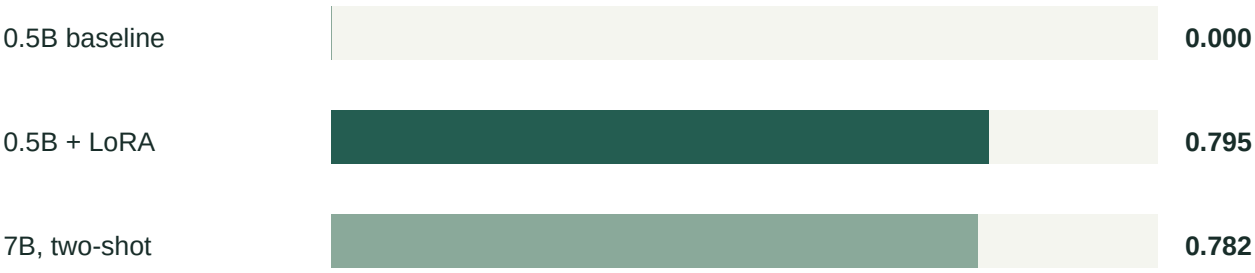
A GPU training attempt stopped after 224 steps with a Metal “Impacting Interactivity” error. The final run restarted from the original base on CPU; the failed GPU weights were not reused. It completed all 840 steps with compilation disabled. The trainer reported 6008.016 seconds. All 56 saved adapter tensors were reloaded and checked for exact equality before held-out evaluation.

The validation pool contains 40 records; each loss check sampled two batches rather than the full pool. No test records were used in training, the two few-shot demonstrations, or hyperparameter selection. One final training seed was completed, so variability across training runs is not measured. The adapter includes the base model’s Apache 2.0 license and modification notice; base weights are downloaded separately.

4. Evaluation and results

The protocol was frozen before the final test: 80 inputs per approach, deterministic decoding (temperature 0, seed 42), maximum 768 generated tokens, identical output schema, and no response repair. The longest reference output is 303 tokenizer tokens. Strict JSON parsing rejects prose and code fences; schema-invalid responses count as empty extractions.

Content matching is one-to-one and order-independent. Type, target, operator, value, unit, scope, and weather condition must match after documented case/spacing/Unicode normalization. Micro precision = TP/(TP+FP), recall = TP/(TP+FN), and F1 = 2TP/(2TP+FP+FN). Hard/soft priority is scored separately on matched content. Exact-record accuracy additionally requires the full set, priorities, and clarification flags to be correct.



Approach	P / R / F1	Exact	Valid JSON	Mean sec.
0.5B baseline	0.000 / 0.000 / 0.000	0/80	0.0%	19.42
0.5B + LoRA	0.856 / 0.742 / 0.795	57/80	97.5%	3.93
7B two-shot	0.808 / 0.758 / 0.782	51/80	93.8%	30.01

Uncertainty and measurement limits

Approximate 95% percentile bootstrap intervals for F1 are baseline: 0.000-0.000; LoRA: 0.667-0.900; 7B: 0.615-0.887. The 2,000 resamples draw reference scenarios, keeping all four paraphrases together. Only 20 independent reference groups are available; these intervals describe this synthetic sample, not real-world language variation.

Inference ran on the Apple M2 GPU with 16 GB unified memory. Wall time includes prompt processing and the first generation; loading is recorded separately. The 7B prompt is longer and its weights are quantized. Capacity, precision, prompting, and output length differ. Low schema validity can dominate F1: the strict baseline measures format compliance and content extraction together, not language understanding alone.

5. Error analysis

Family-level scores show which capabilities remain weak. Each family has eight test inputs, formed from just two reference scenarios; differences should be read as diagnostic evidence, not stable rankings. All raw responses, including failures, are retained in the repository.

Family	Baseline F1	LoRA F1	7B F1
place include	0.000	0.800	0.769
place exclude	0.000	1.000	1.000
hotel time	0.000	0.375	0.750
walking limit	0.000	0.750	0.750
walking minimize	0.000	1.000	1.000
ambiguous time	0.000	1.000	0.125
ambiguous walking scope	0.000	0.000	0.000
near lunch	0.000	1.000	1.000
weather alternative	0.000	1.000	0.714
combined	0.000	0.800	0.947

Two observed LoRA failures

ambiguous_walking_scope_00_0: My maximum walking time is 15 minutes.

Reference: {"type": "walking", "target": "walking_duration", "operator": "<=", "value": 15, "priority": "hard", "needs_clarification": true, "condition": null, "unit": "minutes", "scope": null}
Prediction: {"type": "walking", "target": "walking_duration", "operator": "<=", "value": 15, "priority": "hard", "needs_clarification": false, "condition": null, "unit": "minutes", "scope": "per_segment"}

hotel_time_03_0: Neither before nor after 18:45: that is when I must arrive at the hotel.

Reference: {"type": "time", "target": "hotel_return", "operator": "=", "value": "18:45", "priority": "hard", "needs_clarification": false, "condition": null}
Prediction: {"type": "time", "target": "hotel_return", "operator": "<=", "value": null, "priority": "hard", "needs_clarification": true, "condition": null}

The adapted model matched 95 constraint contents, produced 16 false positives, and missed 33. Priority accuracy was 96.8% on the matched constraints only. That conditional percentage must be read together with recall: it excludes missed conditions. No post-test repair, target relabeling, or extra tuning was used to improve the reported scores.

6. London demonstration

The browser demo shows extracted conditions and simple plan edits. Live mode runs the 0.5B LoRA model; offline mode clearly replays actual validation predictions. Editing the request selects live mode. Schema errors are shown without repair.

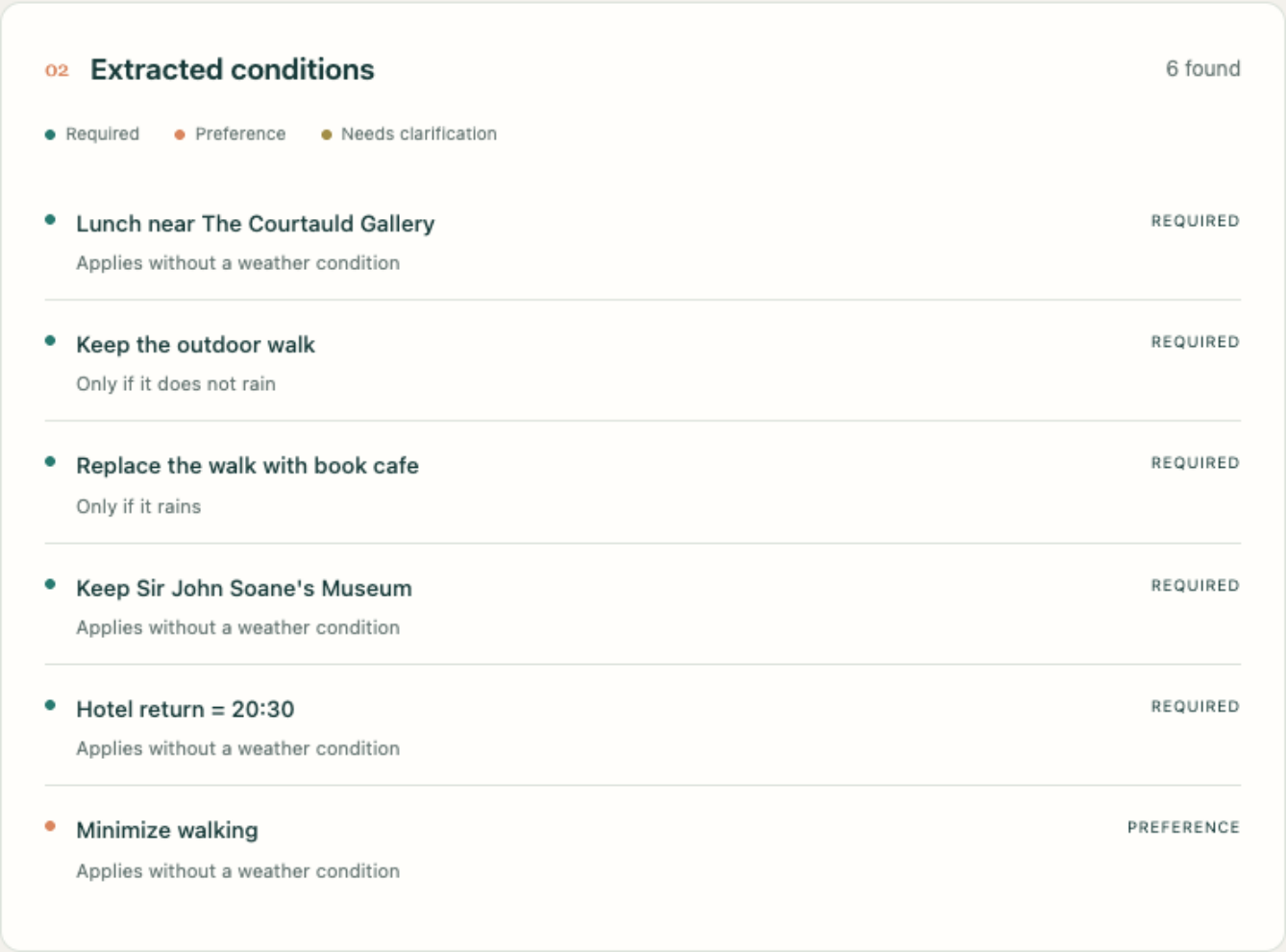


Figure 1. Actual application output from a saved validation prediction. This demonstration example is separate from the held-out test set.

Forecasts and conditional actions

Open-Meteo / ECMWF IFS 0.25-degree forecasts [3] select rain/dry alternatives. Rain and showers describe the preceding hour: a 14:00-15:00 walk uses the bucket ending at 15:00. The application threshold is 0.1 mm/hour. Partial hours are handled conservatively; missing coverage leaves both branches pending. Forecast timestamps and simulated weather are visible.

Place data and interface verification

Six OpenStreetMap locations [4] provide reference coordinates. Nearby means straight-line distance; hours and availability are unverified. Book cafe is a category placeholder. Routes and walking caps are not verified. All 25 automated tests passed; browser checks confirmed rain/dry switching, six conditions, and a mobile layout without horizontal overflow or browser errors.

7. Discussion and reproducibility

On the frozen synthetic test, LoRA changed the small model's content F1 by +79.5 percentage points relative to its untuned baseline. The exact-record score of 71.25% shows that structured extraction still fails on a substantial fraction of requests. Missing-scope and time-operator errors matter because an incorrect interpretation can change the resulting plan. A production system would require broader, independently annotated requests, additional training seeds, and explicit user confirmation of unresolved conditions.

This project demonstrates the complete experimental path: an extraction contract, a documented 400-record corpus, supervised Transformer adaptation, two prompting comparisons, strict held-out scoring, error inspection, and a working conditional demonstration. The weather and place components make the outputs inspectable; they do not substitute for the NLP evaluation.

Reproduce and inspect

The public repository contains source code, the frozen dataset and split IDs, pinned model revisions, the LoRA artifact, raw predictions, metric summaries, and error analysis. The README gives installation and demo commands. Saved examples run without model downloads; live inference uses MLX on Apple Silicon. Base weights and a Python environment are excluded from the submission ZIP. A separate reproduction entry point writes new comparison outputs, preserving the submitted measurements.

github.com/hjarchivenote/travel-constraint-extraction

AI assistance and review status

Codex assisted with topic refinement, implementation, synthetic example and target generation, consistency checks, experiment execution, error analysis, interface construction, and report/presentation drafting. The reported experiments executed the stated models; raw outputs were not manually corrected. Dataset annotations are AI-reviewed, with zero human-verified records. This disclosure does not claim independent human validation or sole unaided implementation. The author should verify the interpretation and comply with the course's AI-assistance policy before submission.

References and source terms

[1] Qwen et al. Qwen2.5 Technical Report (2024). arxiv.org/abs/2412.15115. Model revisions and licenses are recorded in experiments/protocol.json and artifacts/qwen05_lora/.

[2] Hu et al. LoRA: Low-Rank Adaptation of Large Language Models (2021). arxiv.org/abs/2106.09685.

[3] Open-Meteo, ECMWF API documentation and pricing/terms, accessed 29 Oct 2026. open-meteo.com/en/docs/ecmwf-api; open-meteo.com/en/pricing. Forecast data: CC BY 4.0; Open-Meteo / ECMWF attribution retained. Free API used for non-commercial education.

[4] OpenStreetMap contributors. OpenStreetMap data and copyright. openstreetmap.org/copyright. ODbL attribution and source metadata are retained in data/reference/london_places.json.

Project rights and third-party terms: LICENSE_SCOPE.md and THIRD_PARTY_NOTICES.md. No additional blanket reuse license is asserted for the original project materials.